

Plan de Rediseño — CryptolA Trading Bot

Fecha: 23/06/2026
Estado actual: -321 USDT (-3.56%) en 90 ciclos | 3 operaciones | 96% ESPERAR

1. Diagnóstico Honesto del Sistema Actual

Problemas críticos identificados en la DB

Problema	Evidencia	Impacto
Bot en posición con pérdida sin vender	38 ciclos seguidos diciendo ESPERAR con P&L negativo	Pérdida acumulada
Agentes no conocen el negocio	Solo analizan mercado, no saben precio de compra ni P&L	Decisiones sin contexto
Compra inmediata después de pérdida	Vendió con -321 USDT y 30 min después compró de nuevo	Pérdidas compuestas
96% de ciclos dicen ESPERAR	162/168 ciclos = ESPERAR	Bot inactivo
Sin GPU en contenedores	DeviceRequests: null	Respuestas lentas (CPU)
Disco lleno al 100%	98GB de 480GB usados (LVM sin expandir)	Riesgo de crash

Problema raíz (lo que planteaste desde el inicio)

"El agente que analiza nunca sabe cuánto pagué por lo que tengo"

El motor actual **sí** pasa `ctx_posicion` a los agentes (línea 727 de `motor_real.py`), pero el problema es que:

- 1. El agente técnico recibe el contexto pero su prompt lo ignora — solo analiza velas
- 2. No hay lógica de "si estoy en pérdida y el precio está por encima de compra → vender"
- 3. No hay cooldown después de una pérdida
- 4. La decisión de venta con pérdida queda 100% en manos del SL fijo (2.5%)

2. Arquitectura Nueva — Principios

Principio 1: El negocio manda sobre el mercado

ANTES: Analizar mercado → decidir
AHORA: Estado del negocio → filtrar → analizar mercado → decidir

Principio 2: Reglas deterministas primero, IA después

La IA es lenta y costosa. Las reglas de negocio son instantáneas y confiables.

Prioridad de decisión:

1. Stop-Loss fijo → venta automática (sin IA)
2. Take-Profit fijo → venta automática (sin IA)
3. Trailing Stop → venta automática (sin IA)
4. Precio > compra + X% → venta automática (sin IA) ← NUEVO
5. Cooldown post-pérdida → bloquear compra (sin IA) ← NUEVO
6. IA técnico + fundamental → compra/venta

Principio 3: Un solo archivo de entrada, estructura clara

```
bot/
├── config.py           ← toda la configuración en un lugar
├── run.py             ← punto de entrada único
├── core/
│   ├── estado.py      ← estado del bot (posición, saldo, historial)
│   ├── reglas.py      ← reglas deterministas de negocio
│   ├── mercado.py     ← datos de Binance + indicadores
│   └── loop.py         ← ciclo principal
├── agentes/
│   ├── tecnico.py     ← agente técnico (GPU0)
│   ├── fundamental.py ← agente fundamental (GPU1)
│   └── riesgo.py       ← gestor de riesgos (GPU2)
├── exchange/
│   └── binance.py     ← compra/venta real y simulada
├── db/
│   └── postgres.py    ← persistencia en PostgreSQL
├── tests/
│   └── verificar.py   ← diagnóstico rápido
```

3. Lógica de Trading Correcta

3.1 Reglas de VENTA (deterministas, sin IA)

```
def evaluar_salida(precio_actual, estado):
    pc = estado.precio_compra
    pnl = (precio_actual - pc) / pc * 100

    # R1: Stop-Loss fijo
    if precio_actual <= estado.sl_precio:
        return "VENTA_SL", f"SL activado: {pnl:.2f}%"

    # R2: Take-Profit fijo
    if precio_actual >= estado.tp_precio:
        return "VENTA_TP", f"TP activado: {pnl:.2f}%"
```

```
# R3: Trailing Stop
if estado.trailing_activo and precio_actual <= estado.trailing_precio:
    return "VENTA_TRAILING", f"Trailing: {pnl:.2f}%"

# R4: NUEVO – Si precio > compra y tendencia bajista → vender con ganancia
mínima
if pnl >= 0.3 and indicadores.tendencia == "BAJISTA":
    return "VENTA_DEFENSIVA", f"Ganancia {pnl:.2f}% con tendencia bajista"

# R5: NUEVO – Si llevamos demasiado tiempo en pérdida → cortar
if estado.ciclos_en_posicion > 20 and pnl < -1.0:
    return "VENTA_TIMEOUT", f"Pérdida {pnl:.2f}% por
{estado.ciclos_en_posicion} ciclos"

return None, ""
```

3.2 Reglas de COMPRA (deterministas, sin IA)

```
def puede_comprar(estado):
    # Bloqueo 1: Ya estoy en posición
    if estado.en_posicion:
        return False, "Ya en posición"

    # Bloqueo 2: NUEVO – Cooldown después de pérdida
    if estado.ultima_operacion_fue_perdida:
        minutos_desde_perdida = (ahora() - estado.ts_ultima_venta).seconds / 60
        if minutos_desde_perdida < COOLDOWN_POST_PERDIDA_MIN:
            return False, f"Cooldown post-pérdida: {minutos_desde_perdida:.0f}
min"

    # Bloqueo 3: Máximo de operaciones por día
    if estado.operaciones_hoy >= MAX_OPERACIONES_DIA:
        return False, "Máximo de operaciones del día alcanzado"

    return True, ""
```

3.3 Prompt correcto para los agentes

El problema actual: el prompt le dice al agente "analiza el mercado" pero no le dice qué hacer con la posición.

Prompt nuevo para agente técnico:

Eres un trader profesional. Tu objetivo es GANAR DINERO, no solo analizar.

SITUACIÓN ACTUAL:

- Precio BTC: \$62.358
- Compré a: \$62.454 (hace 38 ciclos = ~5 horas)
- P&L actual: -0.15% (-\$15 USDT)
- Stop-Loss: \$60.893 | Take-Profit: \$65.577

PREGUNTA CONCRETA:

¿Debo VENDER ahora (asumir pérdida pequeña) o MANTENER esperando recuperación?

Indicadores técnicos: RSI=44, MACD=bajista, EMA=lateral

Responde con JSON: {"accion": "VENTA|ESPERAR", "confianza": 0-100, "razon": "..."}

4. Hoja de Ruta — Pasos en Orden

FASE 1 — Infraestructura (1-2 horas)

- ☐ **1.1** Expandir LVM para usar los 480GB del disco
- ☐ **1.2** Reconfigurar contenedores Docker con imagen ROCm + GPU dedicada
- ☐ **1.3** Verificar que los modelos responden en < 5 segundos con GPU

FASE 2 — Nuevo bot desde cero (4-6 horas)

- ☐ **2.1** Crear carpeta `bot/` con estructura limpia
- ☐ **2.2** `bot/core/estado.py` — estado del bot con historial de operaciones
- ☐ **2.3** `bot/core/reglas.py` — reglas deterministas (SL, TP, cooldown, timeout)
- ☐ **2.4** `bot/core/mercado.py` — datos de Binance simplificados
- ☐ **2.5** `bot/agentes/tecnico.py` — prompt mejorado con contexto de negocio
- ☐ **2.6** `bot/agentes/fundamental.py` — análisis de noticias
- ☐ **2.7** `bot/core/loop.py` — ciclo principal limpio
- ☐ **2.8** `bot/run.py` — punto de entrada único
- ☐ **2.9** `bot/db/postgres.py` — persistencia

FASE 3 — Testing en testnet (2-3 horas)

- ☐ **3.1** Correr 20 ciclos en modo simulado y verificar decisiones
- ☐ **3.2** Verificar que el cooldown post-pérdida funciona
- ☐ **3.3** Verificar que vende cuando P&L > 0 y tendencia baja
- ☐ **3.4** Activar MODO_REAL=true con BINANCE_TESTNET=true

FASE 4 — Monitoreo (continuo)

- ☐ **4.1** Dashboard simple con estado actual
 - ☐ **4.2** Alertas cuando se ejecuta una operación
 - ☐ **4.3** Reporte diario de P&L
-

5. Parámetros Recomendados

```
# Más agresivo para capturar movimientos
STOP_LOSS_PCT=1.5          # era 2.5 — cortar pérdidas más rápido
TAKE_PROFIT_PCT=2.0        # era 5.0 — tomar ganancias más seguido
TRAILING_STOP_ACTIVACION_PCT=1.0  # era 2.0
```

```
TRAILING_STOP_PROTECCION_PCT=0.3    # era 0.5

# Nuevos parámetros
COOLDOWN_POST_PERDIDA_MIN=60        # esperar 1 hora después de pérdida
VENTA_TIMEOUT_CICLOS=15              # vender si llevo 15 ciclos en pérdida
VENTA_MINIMA_GANANCIA_PCT=0.3        # vender si hay 0.3% de ganancia y tendencia
baja
CONFIANZA_MIN_COMPRA=60              # era 55 — más selectivo para entrar
```

6. Lo que NO cambiar

- La conexión a Binance Testnet funciona bien
- La estructura de la DB (billetera, ciclos_observacion) es correcta
- El sistema de logging a CSV es útil
- Los 3 agentes IA son el enfoque correcto — solo necesitan mejores prompts

7. Próximo paso inmediato

Empezar por FASE 2.1 — crear **bot/** con la nueva estructura.

El código nuevo coexiste con el viejo hasta que esté probado.

Cuando funcione, se reemplaza **run_bot.py** para apuntar al nuevo.

```
# Para iniciar el nuevo bot (cuando esté listo):
python bot/run.py

# Para el bot actual (mientras tanto):
python run_bot.py
```